Manuel Développeur — Backpack Hero (version Java)

0. Objectif du document

Ce manuel décrit l'architecture et l'organisation interne du projet **Backpack Hero** tel qu'implémenté dans le code Java fourni.
Il s'adresse à un développeur qui reprend le projet, souhaite comprendre **où se trouve la logique**, **comment les écrans s'enchaînent**, et **quels sont les points d'extension** (ajout d'items, ennemis, salles, événements…).

---

1. Vue d'ensemble de l'architecture

1.1. Découpage logique

Le projet est organisé autour d'un découpage classique :

- **Modèle (Model)** : règles métier (combat, héros, inventaire, donjon, items, ennemis).
- **Vues (View)** : rendu + interactions utilisateur (clavier / souris).
- **Contrôleur (Controller)** : boucle de jeu et gestion des transitions entre vues.

Le code actuel conserve cette séparation, mais la mise en page (centrage, métriques, tailles de cellules…) est majoritairement calculée **directement dans les vues**, à partir des dimensions width/height passées à draw().

1.2. Packages principaux observés

- hero : état du joueur et progression.
- item : inventaire + items (Shape, Curse, ManaStone, etc.).
- enemy : définition et instances d'ennemis.
- combat / main : orchestration du combat et actions.
- map : donjon / étages / salles.
- display : vues et rendu graphique (DungeonView, CombatView, BackPackView…).
- service : services utilitaires (ex : ScoreManager).

---

2. Modèle

2.1. Hero (package hero)

2.1.1. Rôle

Hero représente l'état du joueur : statistiques, ressources, progression et inventaire.
C'est l'entité centrale que manipulent **Dungeon**, **Combat**, et les vues.

2.1.2. Responsabilités attendues (à retrouver dans le code)

- Statistiques : hp/maxHp, protection.
- Ressources : energy/maxEnergy et éventuellement mana/maxMana (selon implémentation).
- Progression : level, exp, expToLevel.
- Position sur carte : (row, col) utilisée par DungeonView.
- Statuts : via StatusManager (ex : DODGE, HASTE, SLOW…).
- Inventaire : via BackPack (accès direct dans les vues et le combat).

2.1.3. Points d'extension

- Équilibrage : valeurs de gain d'XP, énergie, scaling.
- Progression : attribution de points de déverrouillage au BackPack.

2.2. BackPack (package item)

2.2.1. Rôle

BackPack gère l'inventaire spatial du héros sous forme de **grille** et applique les règles de placement, rotation, overflow et malédictions.

2.2.2. Structure interne

- Dimensions max : MAX_WIDTH = 9, MAX_HEIGHT = 5.
- Grille logique : Item[][] grid.
- Masque d'activation : boolean[][] activeCells (cases réellement jouables).
- Items en grille : List<Item> items.
- Items hors grille (overflow) :
    - List<Item> outsideItems
    - Map<Item, Point> outsidePositions

2.2.3. Règles principales

A) Placement normal

add(item) applique une stratégie en 3 étapes :

1. **Stack de l'or** (si Gold)
2. Sinon tentative de placement automatique via canFit(...)
3. Sinon placement en **overflow** (placeOutside)

B) Déverrouillage de cases

- availableUnlocks représente les points disponibles.
- getUnlockableCells() retourne les cases verrouillées adjacentes à une case active.
- unlock(x,y) active une case si elle est déverrouillable et consomme un point.

C) Rotation

- Rotation en grille avec rollback si collision (rotate(item)).
- Rotation en overflow avec recalcul du point d'ancrage (rotateOutsideItem(item)).

D) ManaStone (connectivité)

Le sac supporte une logique de "réseau" :

- recherche de ManaStone connectées à un item,
- consommation répartie sur plusieurs pierres,
- refill global via refillManaStones().

2.2.4. Points d'extension

- Modifier les formes de départ (init_backpack()).
- Ajuster les règles d'overflow (perte, tri, placement visuel).
- Ajouter des règles de placement spécifiques à certains items.

2.3. Items, Shape, Factory (package item)

2.3.1. Item (interface scellée)

Le système d'objets est unifié par une interface (scellée) qui impose :

- identifiants : id, name, rarity
- logique inventaire : shape, withShape(...), isDraggable, isRotatable
- économie : getValue, isSellable
- coûts : energyCost, manaCost
- UI : getStatDescription, getTexturePath, overlay optionnel

- combat : combatAction() → Optional<CombatAction>

Conséquence pratique : les vues et le combat peuvent manipuler des items de façon homogène.

2.3.2. Shape

Représente l'occupation en 2D :

- matrice booléenne
- accès sécurisé get(x,y)
- rotation 90° horaire rotate()
- fabrique rectangle(w,h)

2.3.3. ItemFactory

Centralise :

- génération d'IDs
- tables de loot (coffres, ennemis, shop)
- répartition or/équipement
- raretés et éventuels effets de statuts

2.3.4. Consommables

Les consommables sont des items à usage unique :

- coût (souvent énergie 1, mana 0)
- appliquent un effet au héros
- sont supprimés ensuite du BackPack

2.4. Donjon et génération (package map)

2.4.1. Dungeon

- contient une List<Floor>
- expose addFloor, getFloor(index), getNumberOfFloors()

2.4.2. Floor

- grille 2D Room[][]
- dimensions par défaut (souvent 5×11 via createDefaultFloor())

- accès via getRoom(row,col) avec contrôle des bornes

2.4.3. Room

Une salle porte :

- ses coordonnées (row, col)
- son RoomType
- éventuellement un Door et/ou un Treasure

2.4.4. RoomType

Enum des types fonctionnels :
START, ENEMY, MERCHANT, HEALER, EMPTY, CORRIDOR, DOOR, TREASURE, EVENT, EXIT
Chaque type peut fournir un sprite via getImagePath().

2.4.5. Génération (InitFloor)

Le générateur observé suit une stratégie simple :

- initialise une grille de EMPTY
- place un ensemble de salles spéciales
- relie les salles par des corridors (chemins “carvés”)
- place une porte (souvent proche de EXIT) pour verrouiller la sortie

---

2.5. Ennemis (package enemy)

2.5.1. EnemyType (template)

EnemyType décrit l’ennemi “par classe” (données et IA) :

- stats : maxHp, attackDamage, protectionGain, expReward
- IA : planAction() renvoie une liste d’EnemyAction
- texture : getTexturePath()
- validation : EnemyType.validate(type)

2.5.2. EnemyState (instance runtime)

EnemyState contient l’état pendant un combat :

- hp, protection, statusManager
- dégâts : la protection absorbe d’abord, dodge possible si statut DODGE
- modifications : heal, dégâts directs, protection modulée par HASTE/SLOW

2.5.3. EnemyList

Registre/pool d'ennemis :

- liste statique des EnemyType
- tirage aléatoire + création d'un EnemyState

2.6. Combat (packages main / combat)

2.6.1. CombatAction

Une action d'item en combat :

- needsTarget()
- energyCost()
- execute(combat, targetIndex)

2.6.2. Combat (orchestrateur)

Le combat gère :

- spawn d'ennemis (souvent 1 à 3)
- tour du héros : reset, ticks de statuts, utilisation d'items
- tour des ennemis : exécution des EnemyAction planifiées
- loot / XP par ennemi mort + résumé
- malédictions : création + décision + placement forcé

---

3. Vues (package display)

3.1. Interface commune : GameView

Chaque vue implémente une interface commune :

- draw(Graphics2D g, int width, int height)
- handlePointerEvent(PointerEvent event)
- handleKeyBoardEvent(KeyboardEvent event)
- updateLogic()
- isDone()
- nextView()

Principe : le contrôleur n'a pas besoin de connaître le type concret de la vue, il pilote uniquement via cette interface.

3.2. DungeonView — exploration

3.2.1. Rôle

Affiche l'étage courant et gère :

- le déplacement du héros
- les interactions sur salles
- les transitions (combat, marchand, healer, event…)

3.2.2. Déplacement

- clic sur une case → calcul de chemin BFS (findPath)
- déplacement animé par pas (MOVE_DELAY)
- à l'arrivée : enterRoom(room) déclenche la logique du type de salle

3.2.3. Prompts YES/NO

Deux interactions bloquantes :

- **DOOR** : nécessite une Key dans le sac, consommée à l'ouverture
- **EXIT** : confirmation avant de changer d'étage

3.2.4. Entrée en salle

Exemples observés :

- ENEMY : nettoyage overflow + refill mana stones + transition CombatView + salle devient CORRIDOR
- TREASURE : loot + ajout au sac + salle devient CORRIDOR
- MERCHANT : transition MerchantView + salle devient CORRIDOR
- EVENT : overlay événement + salle devient CORRIDOR

3.3. CombatView — combat tour par tour

3.3.1. Interaction principale

- clic item → sélection
- clic ennemi → cible

- si l’action nécessite une cible : exécution sur l’ennemi ciblé
- bouton / touche “fin du tour” → endHeroTurn()

3.3.2. Malédictions : flow explicite

Quand une curse est déclenchée :

1. CombatView affiche une décision : **ACCEPT / REFUSE**
2. si acceptation → transition vers CursePlacementView
3. tant que la malédiction n’est pas placée, le flow de combat est bloqué

3.4. CursePlacementView — placement forcé

3.4.1. Rôle

Écran dédié au placement d’une malédiction :

- preview sous la souris
- vérification des cases actives
- clic → tentative de placement
- destruction des items recouverts via BackPack.placeCurse

3.4.2. Sortie

Quand la malédiction est placée : retour vers la vue parent (souvent CombatView).

3.5. BackPackView — gestion inventaire

3.5.1. Rôle

Affiche :

- grille max 9×5
- masque de cases actives + cases déverrouillables (si points disponibles)
- zone “outside” (overflow)

3.5.2. Interactions

- clic case verrouillée déverrouillable → unlock(x,y)
- sélection item → infobox
- drag & drop (uniquement si isDraggable())

- rotation (R) si isRotatable()
- usage consommable (SPACE/U) → application + suppression

---

4. Contrôleur

4.1. GameController

Le contrôleur assure :

- boucle principale (Application.run)
- dispatch des événements clavier/souris vers la vue active
- appel updateLogic() + draw()
- transition via isDone() puis nextView()

---

5. Fonctionnement global

5.1. Démarrage

1. création Hero + BackPack initialisé
2. création Dungeon
3. ajout d'un Floor par défaut si nécessaire
4. affichage DungeonView + placement du héros sur START

5.2. Cycle de jeu

- **Exploration** : BFS + animation
- **Rencontre** : entrée salle (Enemy/Merchant/Healer/Treasure/Event/Door/Exit)
- **Combat** : actions via items (CombatAction), tour ennemis (EnemyAction)
- **Malédictions** : refus dégâts croissants ; acceptation placement forcé
- **Récompenses** : XP + loot ; level-up → points de déverrouillage du sac

---

6. Évolutions majeures depuis la soutenance β (comparatif)

6.1. BackPack : grille fixe + masque actif (au lieu d'une grille qui "grandit")

- Avant : déverrouillage sur une grille plus petite (type 5×7) orientée "progression"
- Maintenant : grille max 9×5 + activeCells[][] + availableUnlocks

6.2. Malédictions : séparation en deux écrans (décision → placement)

- Avant : mode spécial plutôt couplé à la logique combat
- Maintenant : CombatView gère décision, puis CursePlacementView gère placement

6.3. Donjon : connectivité via corridors + BFS (au lieu d'un graphe de voisins)

- Avant : salles reliées par voisins bidirectionnels + génération labyrinthique
- Maintenant : carving de corridors + pathfinding BFS sur la grille (RoomType + portes)

6.4. Ennemis : template/instance (EnemyType + EnemyState) au lieu d'héritage direct

- Avant : Enemy abstrait + classes concrètes
- Maintenant : EnemyType (données + IA) + EnemyState (runtime) + validation

---

7. Points de maintenance / extension

7.1. Ajouter un nouvel ennemi

1. créer un nouveau EnemyType
2. l'ajouter au registre EnemyList
3. fournir getTexturePath() + actions planAction()

7.2. Ajouter un nouvel item

1. ajouter le type dans la hiérarchie scellée Item
2. définir shape, coût, valeur, texture
3. si combat : fournir combatAction()
4. ajouter dans les pools ItemFactory

7.3. Ajouter un nouveau type de salle

1. ajouter un RoomType
2. ajouter le sprite dans getImagePath()
3. implémenter la réaction dans DungeonView.enterRoom(...)
4. optionnel : intégrer à la génération InitFloor